

Windowing and Clipping

5.1 Introduction

Typically, a **graphics** package allows us to specify which part of a defined picture is to be displayed and where that part is to be displayed on the display device. Furthermore, the package also provides the use of the scaling, translation and rotation techniques described in the previous chapter to generate a variety of different views of a single picture. We can generate different view of a picture **by** applying the appropriate scaling and translation. While doing this, we have to identify the visible part of the picture for inclusion in the display image. This selection process is not straight forward. Certain lines may lie partly inside the visible portion of the picture and partly outside. These lines cannot be omitted entirely from the display image because the image would become inaccurate. This is illustrated in Fig. 5.1. The process of selecting and viewing the picture with different views is called **windowing**, and a process which divides each element of the picture into its visible and invisible portions, allowing the invisible portion to be discarded is called **clipping**.

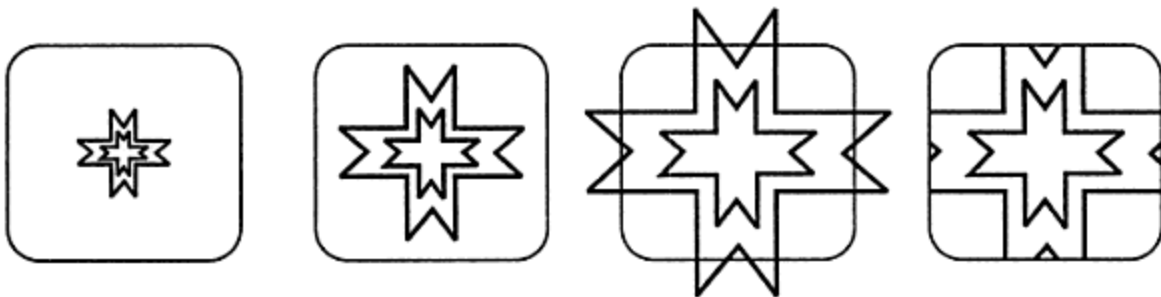


Fig. 5.1

In this chapter we are going to discuss the concepts involved in windowing and various clipping algorithms

5.2 Viewing Transformation

We know that the picture is stored in the **computer** memory using any convenient cartesian co-ordinate system, referred to as **world co-ordinate system** (WCS). However, when picture is displayed on the display device it is measured in **physical device co-ordinate system** (PDCS) corresponding to the display device. Therefore, displaying an image of a picture involves mapping the co-ordinates of the points and lines that form the picture into the appropriate physical device co-ordinate where the image is to be displayed. This mapping of co-ordinates is achieved with the use of co-ordinate transformation known as **viewing transformation**.

The viewing transformation which maps picture co-ordinates in the WCS to display co-ordinates in PDCS is performed by the following transformations :

- Normalization transformation (N) and
- Workstation transformation (W)

5.2.1 Normalization Transformation

We know that, different display devices may have different screen sizes as measured in pixels. Size of the screen in pixels increases as resolution of the screen increases. When picture is defined in the pixel values then it is displayed large in size on the low resolution screen while small in size on the high resolution screen as shown in the Fig. 5.2. To avoid this and to make our programs to be device independent, we have to define the picture co-ordinates in some units other than pixels and use the interpreter to convert these co-ordinates to appropriate pixel values for the particular display device. The device independent units are called the **normalized device co-ordinates**. In these units, the screen measures 1 unit wide and 1 unit length as shown in the Fig. 5.3. The lower left corner of the screen is the origin, and the upper-right corner is the point (1, 1). The point(0.5, 0.5) is the center of the screen no matter what the physical dimensions or resolution of the actual display device may be.

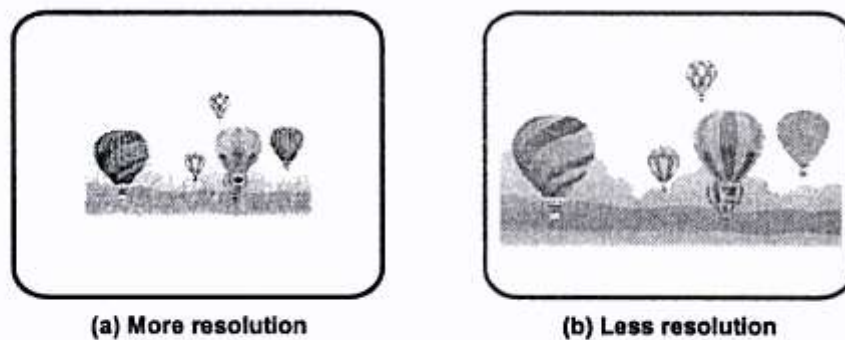


Fig. 5.2 Picture definition in pixels

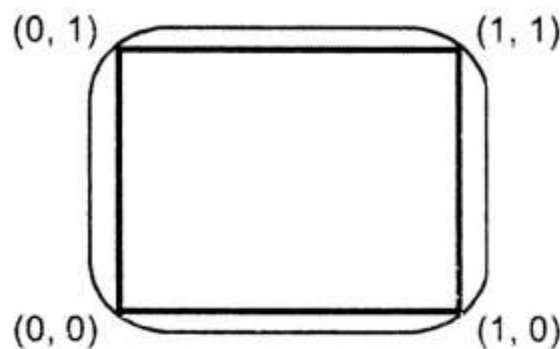


Fig. 6.3 Picture definition in normalized device coordinates

The interpreter uses a simple linear formula to convert the normalized device coordinates to the actual device coordinates.

$$x = x_n \times X_W \quad \dots (6.1)$$

$$y = y_n \times Y_H \quad \dots (6.2)$$

where

x : Actual device x coordinate

y : Actual device y coordinate

x_n : Normalized x coordinate

y_n : Normalized y coordinate

X_W : Width of actual screen in pixels

Y_H : Height of actual screen in pixels.

The transformation which maps the world coordinate to normalized device coordinate is called **normalization transformation**. It involves scaling of x and y, thus it is also referred to as **scaling transformation**.

6.2.2 Workstation Transformation

The transformation which maps the normalized device coordinates to physical device coordinates is called workstation transformation.

The viewing transformation is the combination of normalization transformation and workstation transformations as shown in the Fig. 6.4. It is given as

$$V = N \cdot W \quad \dots (6.3)$$

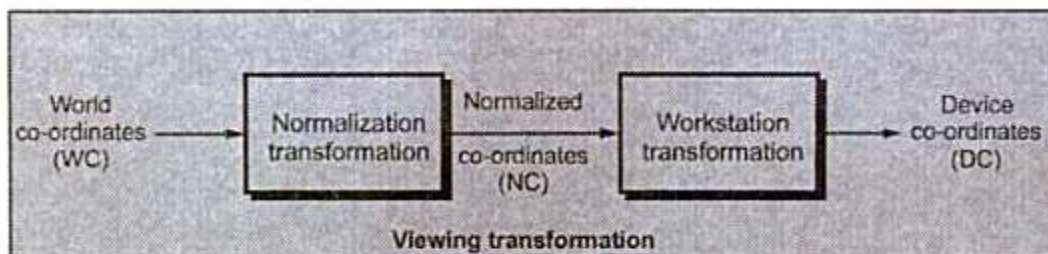


Fig. 5.4 Two dimensional viewing transformation

We know that world co-ordinate system (WCS) is infinite in extent and the device display area is finite. Therefore, to perform a viewing transformation we select a finite world co-ordinate area for display called a **window**. An area on a device to which a window is mapped is called a **viewport**. The window defines what is to be viewed; the viewport defines where it is to be displayed, as shown in the Fig. 5.5.

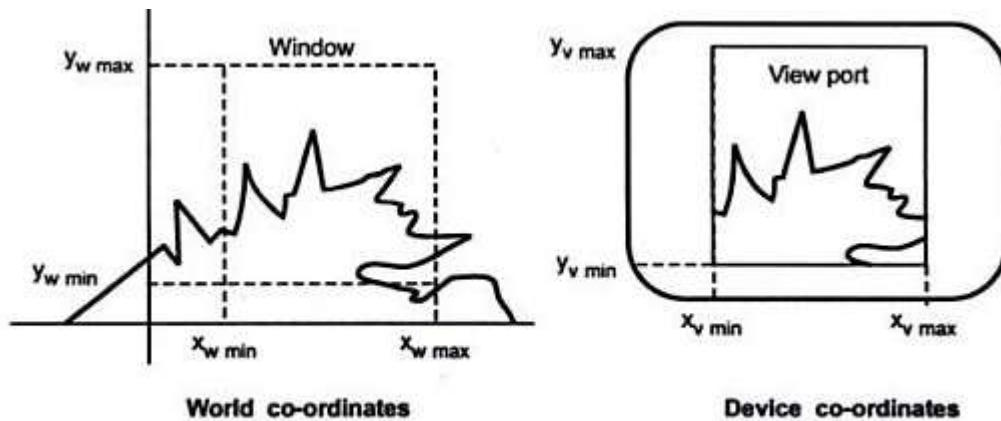


Fig. 5.5 Window and viewport

The window defined in world co-ordinates is first transformed into the normalized device co-ordinates. The normalized window is then transformed into the viewport co-ordinate. This window to viewport co-ordinate transformation is known as workstation transformation. It is achieved by performing following steps :

1. The object together with its window is translated until the lower left corner of the window is at the origin.
2. Object and window are scaled until the window has the dimensions of the viewport.
3. Translate the viewport to its correct position on the screen.

This is illustrated in Fig.6.6.

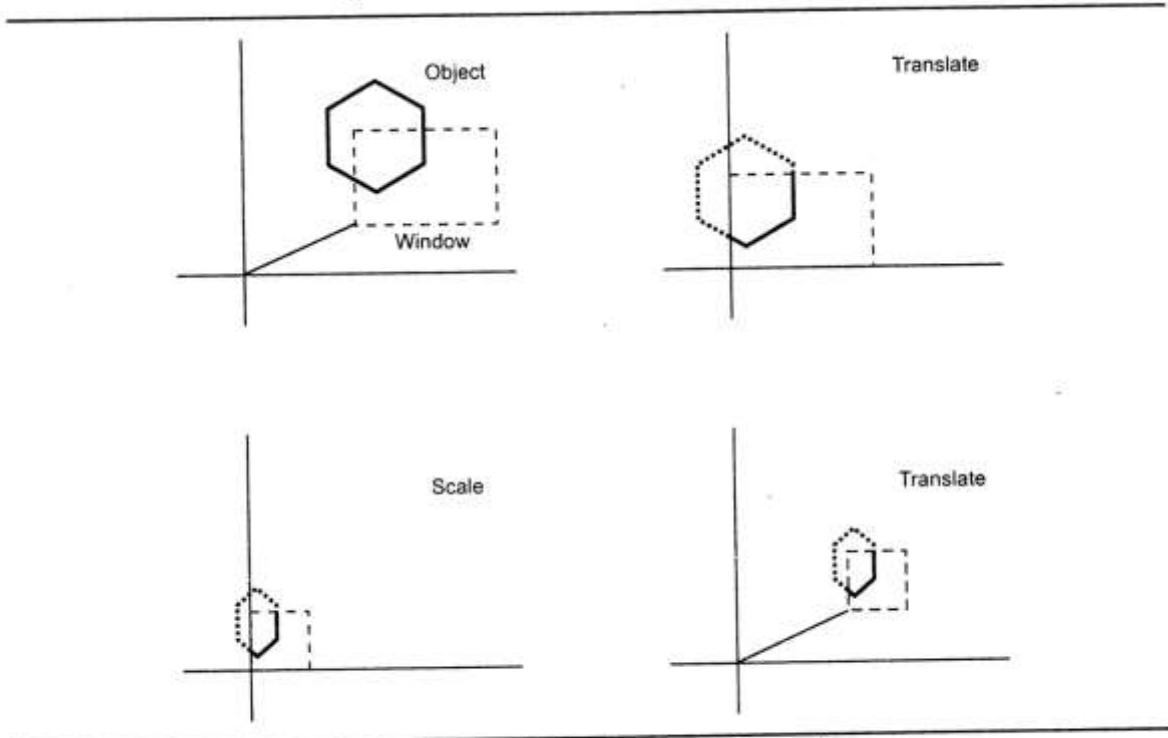


Fig. 6.6 Steps in workstation transformation

Therefore, the workstation transformation is given as

$$W = T \cdot S \cdot T^{-1}$$

... (6.4)

The transformation matrices for individual transformation are as given below :

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -X_{w \min} & -y_{w \min} & 1 \end{bmatrix}$$

$$S = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{where } S_x = \frac{x_{v \max} - x_{v \min}}{x_{w \max} - x_{w \min}}$$

$$S_y = \frac{y_{v \max} - y_{v \min}}{y_{w \max} - y_{w \min}}$$

$$T^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_{v \min} & y_{v \min} & 1 \end{bmatrix}$$

The overall transformation matrix for W is given as

$$\begin{aligned} W &= T \cdot S \cdot T^{-1} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -X_{w \min} & -y_{w \min} & 1 \end{bmatrix} \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_{v \min} & y_{v \min} & 1 \end{bmatrix} \\ &= \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ x_{v \min} - x_{w \min} \cdot S_x & y_{v \min} - y_{w \min} \cdot S_y & 1 \end{bmatrix} \end{aligned}$$

The Fig. 6.7 shows the complete viewing transformation.

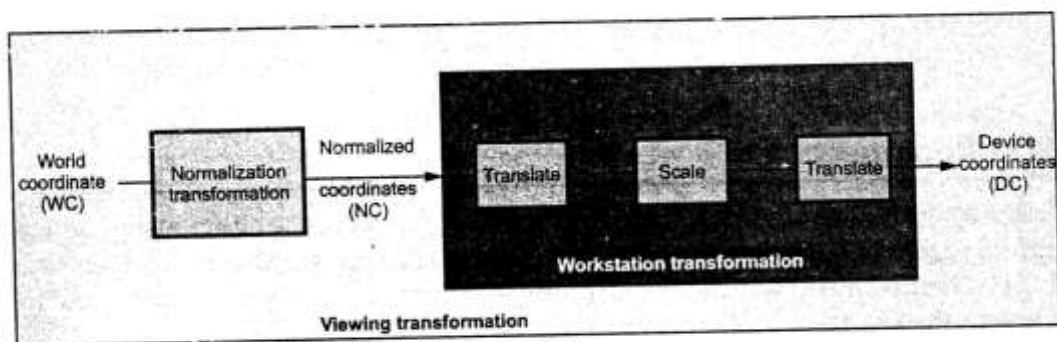


Fig. 6.7 Viewing transformation

Ex. 6.1 : Find the normalization transformation window to viewpoint, with window, lower left corner at (1, 1) and upper right corner at (3, 5) onto a viewpoint with lower left corner at (0, 0) and upper right corner at (1/2, 1/2).

Sol. : Given : Coordinates for window

$$\begin{aligned}x_{w \min} &= 1 & y_{w \min} &= 1 \\x_{w \max} &= 3 & y_{w \max} &= 5\end{aligned}$$

Coordinates for view port

$$\begin{aligned}x_{v \min} &= 0 & y_{v \min} &= 0 \\x_{v \max} &= 1/2 = 0.5 & y_{v \max} &= 1/2 = 0.5\end{aligned}$$

We know that,

$$\begin{aligned}S_x &= \frac{x_{v \max} - x_{v \min}}{x_{w \max} - x_{w \min}} \\&= \frac{0.5 - 0}{3 - 1} \\&= 0.25\end{aligned}$$

$$\begin{aligned}\text{and } S_y &= \frac{y_{v \max} - y_{v \min}}{y_{w \max} - y_{w \min}} \\&= \frac{0.5 - 0}{5 - 1} \\&= 0.125\end{aligned}$$

We know that transformation matrix is given as

$$\begin{aligned}T \cdot S \cdot T^{-1} &= \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ x_{v \min} - x_{w \min} S_x & y_{v \min} - y_{w \min} S_y & 1 \end{bmatrix} \\&= \begin{bmatrix} 0.25 & 0 & 0 \\ 0 & 0.125 & 0 \\ 0 - (1 \times 0.25) & 0 - (1 \times 0.125) & 1 \end{bmatrix} \\&= \begin{bmatrix} 0.25 & 0 & 0 \\ 0 & 0.125 & 0 \\ -0.25 & -0.125 & 1 \end{bmatrix}\end{aligned}$$

5.3.1 Point Clipping

The points are said to be interior to the clipping window if

$$\begin{aligned}x_{w \min} &\leq x \leq x_{w \max} & \text{and} \\y_{w \min} &\leq y \leq y_{w \max}\end{aligned}$$

The equal sign indicates that points on the window boundary are included within the window.

5.3.2 Line Clipping

The lines are said to be interior to the clipping window and hence visible if both end points are interior to the window, e.g., line $P_1 P_2$ in Fig. 5.8. However, if both end points of a line are exterior to the window, the line is not necessarily completely exterior to the window, e.g. line $P_7 P_8$ in Fig. 5.8. If both end points of a line are completely to the right of, completely to the left of, completely above, or completely below the window, then the line is completely exterior to the window and hence invisible. For example, line $P_3 P_4$ in Fig. 5.8.

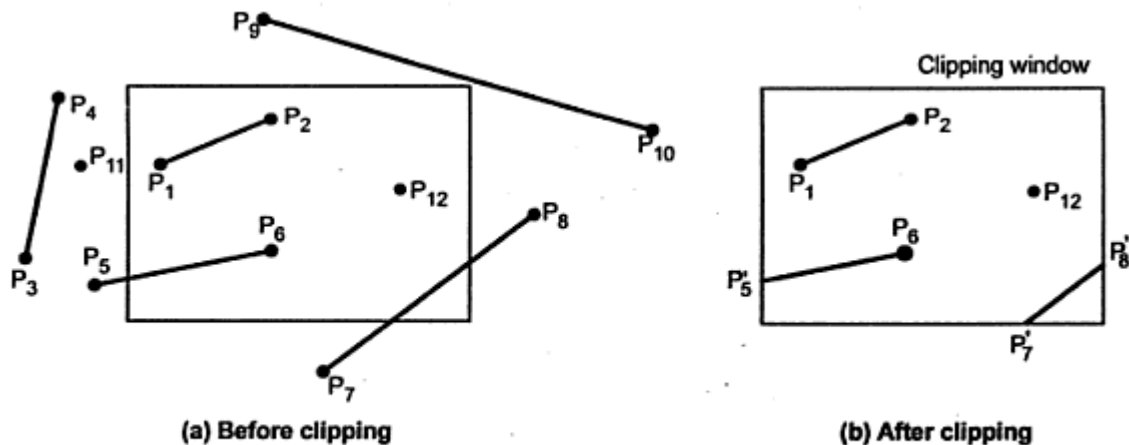


Fig. 5.8

The lines which across one or more clipping boundaries require calculation of multiple intersection points to decide the visible portion of them. To minimize the intersection calculations and to increase the efficiency of the clipping algorithm, initially, completely visible and invisible lines are identified and then the intersection points are calculated for remaining lines. There are many line clipping algorithms. Let us discuss a few of them.

5.3.2.1 Sutherland and Cohen Subdivision Line Clipping Algorithm

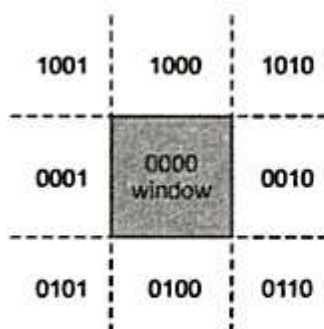


Fig. 5.9 Four-bit codes for nine regions

This is one of the oldest and most popular line clipping algorithm developed by Dan Cohen and Ivan Sutherland. To speed up the processing this algorithm performs initial tests that reduce the number of intersections that must be calculated. This algorithm uses a four digit (bit) code to indicate which of nine regions contain the end point of line. The four bit codes are called **region codes** or **outcodes**. These codes identify the location of the point relative to the boundaries of the clipping rectangle as shown in the Fig. 5.9.

Each bit position in the region code is used to indicate one of the four relative co-ordinate positions of the point with respect to the clipping window : to the left, right, top or bottom. The rightmost bit is the first bit and the bits are set to 1 based on the following scheme :

- Set Bit 1 – if the end point is to the left of
the window
- Set Bit 2 – if the end point is to the right of
the window
- Set Bit 3 – if the end point is below the
window
- Set Bit 4 – if the end point is above the window
- Otherwise, the bit is set to zero.

Once we have established region codes for all the line endpoints, we can determine which lines are completely inside the clipping window and which are clearly outside. Any lines that are completely inside the window boundaries have a region code of 0000 for both endpoints and we trivially accept these lines. Any lines that have a 1 in the same bit position in the region codes for each endpoint are completely outside the clipping rectangle, and we trivially reject these lines. A method used to test lines for total clipping is equivalent to the logical AND operator. If the result of the logical AND operation with two end point codes is not 0000, the line is completely outside the clipping region. The lines that cannot be identified as completely inside or completely outside a clipping window by these tests are checked for intersection with the window boundaries.

Sol.: The Fig. 6.11 shows the clipping window and lines with region codes. These codes are tabulated and end point codes are logically ANDed to identify the visibility of the line in table 6.1.

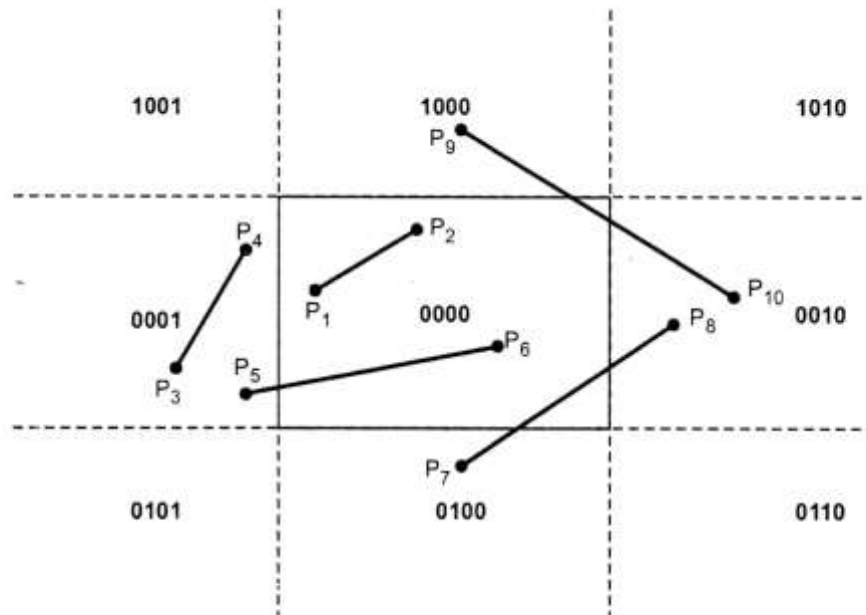


Fig. 6.11

Line	End Point Codes		Logical ANDing	Result
P ₁ P ₂	0000	0000	0000	Completely visible
P ₃ P ₄	0001	0001	0001	Completely invisible
P ₅ P ₆	0001	0000	0000	Partially visible
P ₇ P ₈	0100	0010	0000	Partially visible
P ₉ P ₁₀	1000	0010	0000	Partially visible

Table 5.1

The Sutherland - Cohen algorithm begins the clipping process for a partially visible line by comparing an outside endpoint to a clipping boundary to determine how much of the line can be discarded. Then the remaining part of the line is checked against the other boundaries, and the process is continued until either the line is totally discarded or a section is found inside the window.

This is illustrated in Fig. 5.12.

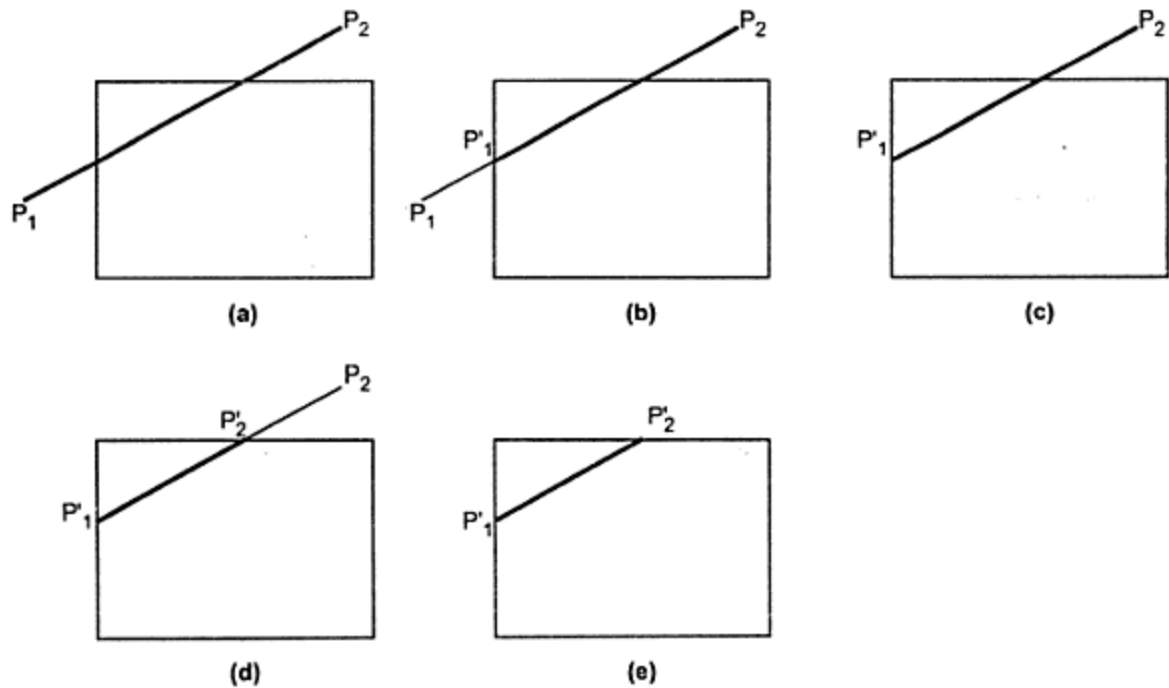


Fig. 5.12 Sutherland-Cohen subdivision line clipping

As shown in the Fig. 5.12, line P_1P_2 is a partially visible and point P_1 is outside the window. Starting with point P_1 , the intersection point P'_1 is found and we get two line segments $P_1 - P'_1$ and $P'_1 - P_2$. We know that, for $P_1 - P'_1$ one end point i.e. P_1 is outside the window and thus the line segment $P_1 - P'_1$ is discarded. The line is now reduced to the section from P'_1 to P_2 . Since P_2 is outside the clip window, it is checked against the boundaries and intersection point P'_2 is found. Again the line segment is divided into two

segments giving $P'_1 - P'_2$ and $P'_2 - P_2$. We know that, for $P'_2 - P_2$ one end point i.e. P_2 is outside the window and thus the line segment $P'_2 - P_2$ is discarded. The remaining line segment $P'_1 - P'_2$ is completely inside the clipping window and hence made visible.

The intersection points with a clipping boundary can be calculated using the slope-intercept form of the line equation. The equation for line passing through points $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ is

$$y = m(x - x_1) + y_1 \quad \text{or} \quad y = m(x - x_2) + y_2 \quad \dots (1)$$

where $m = \frac{y_2 - y_1}{x_2 - x_1}$ (slope of the line)

Therefore, the intersections with the clipping boundaries of the window are given as :

Left : $x_L, y = m(x_L - x_1) + y_1$; $m \neq \infty$

Right : $x_R, y = m(x_R - x_1) + y_1$; $m \neq \infty$

Top : $y_T, x = x_1 + \left(\frac{1}{m}\right)(y_T - y_1)$; $m \neq 0$

Bottom : $y_B, x = x_1 + \left(\frac{1}{m}\right)(y_B - y_1)$; $m \neq 0$

Sutherland and Cohen subdivision line clipping algorithm :

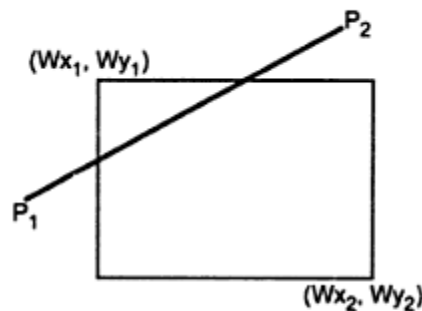


Fig. 5.13

1. Read two end points of the line say $P_1 (x_1, y_1)$ and $P_2 (x_2, y_2)$.
2. Read two corners (left-top and right-bottom) of the window, say (Wx_1, Wy_1) and (Wx_2, Wy_2) .
3. Assign the region codes for two endpoints P_1 and P_2 using following steps :
Initialize code with bits 0000
Set Bit 1 – if $(x < Wx_1)$
Set Bit 2 – if $(x > Wx_2)$
Set Bit 3 – if $(y < Wy_2)$
Set Bit 4 – if $(y > Wy_1)$
4. Check for visibility of line $P_1 P_2$
 - a) If region codes for both endpoints P_1 and P_2 are zero then the line is completely visible. Hence draw the line and go to step 9.
 - b) If region codes for endpoints are not zero and the logical ANDing of them is also nonzero then the line is completely invisible, so reject the line and go to step 9.
 - c) If region codes for two endpoints do not satisfy the conditions in (4a) and (4b) the line is partially visible.
5. Determine the intersecting edge of the clipping window by inspecting the region codes of two endpoints.
 - a) If region codes for both the end points are non-zero, find intersection points P'_1 and P'_2 with boundary edges of clipping window with respect to point P_1 and point P_2 , respectively
 - b) If region code for any one end point is non zero then find intersection point P'_1 or P'_2 with the boundary edge of the clipping window with respect to it.
6. Divide the line segments considering intersection points.
7. Reject the line segment if any one end point of it appears outside the clipping window.
8. Draw the remaining line segments.
9. Stop.

6.3.2.2 Midpoint Subdivision Algorithm

We have seen that, the Sutherland Cohen subdivision line clipping algorithm requires the calculation of the intersection of the line with the window edge. These calculations can be avoided by repeatedly subdividing the line at its midpoint.

Like previous algorithm, initially the line is tested for visibility. If line is completely visible it is drawn and if it is completely invisible it is rejected. If line is partially visible then it is subdivided in two equal parts. The visibility tests are then applied to each half. This subdivision process is repeated until we get completely visible and completely invisible line segments. This is illustrated in Fig. 6.14. (see on next page)

As shown in the Fig. 6.14, line $P_1 P_2$ is partially visible. It is subdivided in two equal parts $P_1 P_3$ and $P_3 P_2$ (see Fig. 6.14(b)). Both the line segments are tested for visibility and found to be partially visible. Both line segments are then subdivided in two equal parts to get midpoints P_4 and P_5 (see Fig. 6.14 (c)). It is observed that line segments $P_1 P_4$ and $P_5 P_2$ are completely invisible and hence rejected. However, line segment $P_3 P_5$ is completely visible and hence drawn. The remaining line segment $P_4 P_3$ is still partially visible. It is then subdivided to get midpoint P_6 . It is observed that $P_6 P_3$ is completely visible whereas $P_4 P_6$ is partially visible. Thus $P_6 P_3$ line segment is drawn and $P_4 P_6$ line segment is further subdivided into equal parts to get midpoint P_7 . Now, it is observed that line segment $P_4 P_7$ is completely invisible and line segment $P_7 P_6$ is completely visible (see Fig. 6.14 (f)), and there is no further partially visible segment.

Midpoint Subdivision Algorithm :

1. Read two endpoints of the line say $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$.
2. Read two corners (left-top and right-bottom) of the window, say (Wx_1, Wy_1) and (Wx_2, Wy_2) .
3. Assign region codes for two end points using following steps :

Initialize code with bits 0000

Set Bit 1 - if $(x < Wx_1)$

Set Bit 2 - if $(x > Wx_2)$

Set Bit 3 - if $(y < Wy_1)$

Set Bit 4 - if $(y > Wy_2)$

4. Check for visibility of line

a) If region codes for both endpoints are zero then the line is completely visible. Hence draw the line and go to step 6.

b) If region codes for endpoints are not zero and the logical ANDing of them is also nonzero then the line is completely invisible, so reject the line and go to step 6.

c) If region codes for two endpoints do not satisfy the conditions in 4a) and 4b) the line is partially visible.

5. Divide the partially visible line segment in equal parts and repeat steps 3 through 5 for both subdivided line segments until you get completely visible and completely invisible line segments.

6. Stop.

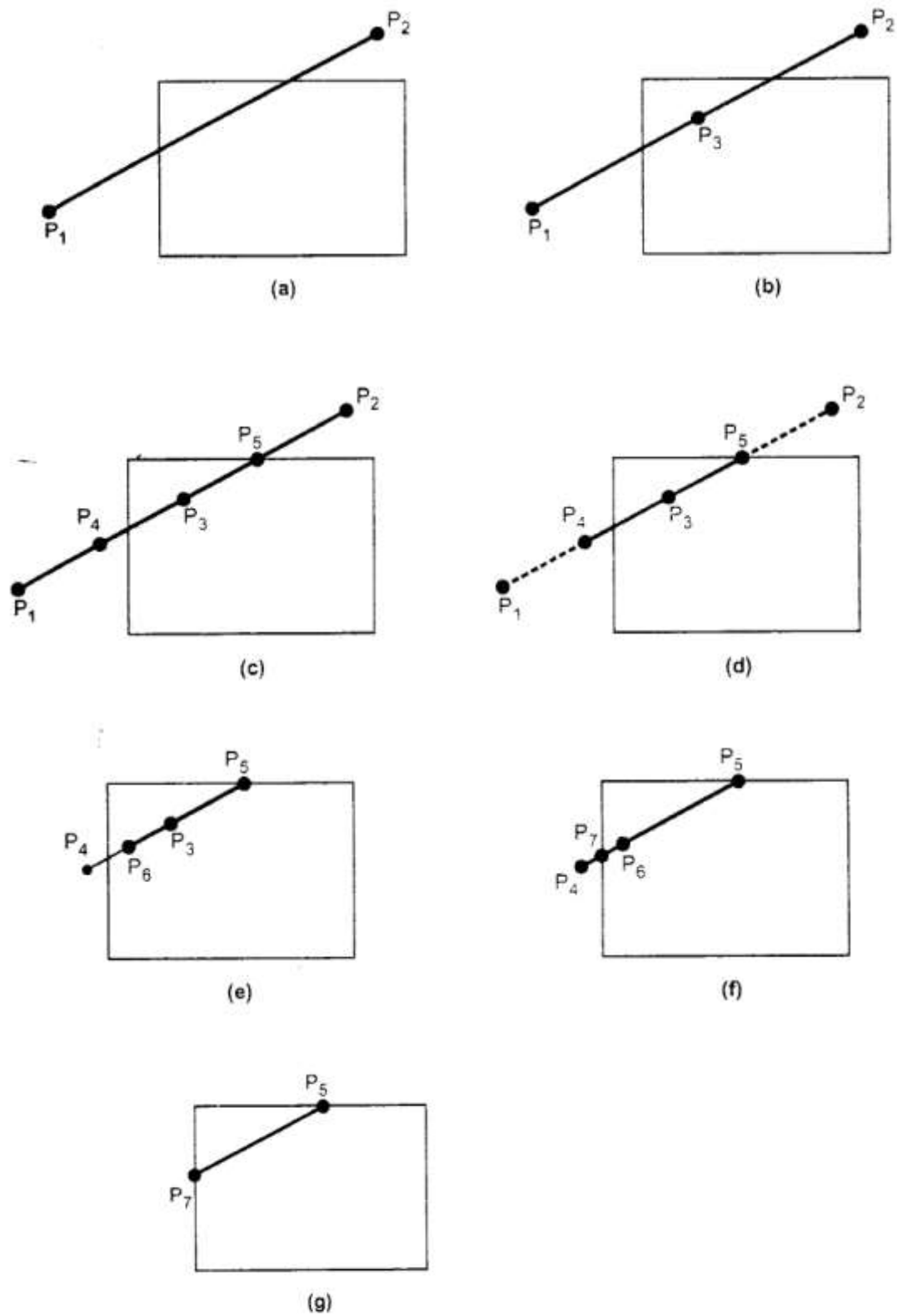


Fig. 6.14 Clipping line with midpoint subdivision algorithm

The midpoint subdivision algorithm requires repeated subdivision of line segments and hence many times it is slower than using direct calculation of the intersection of the line with the clipping window edge. However, it can be implemented efficiently using parallel architecture since it involves parallel operations.

5.5 Polygon Clipping

In the previous sections we have seen line clipping algorithms. A polygon is nothing but the collection of lines. Therefore, we might think that line clipping algorithm can be used directly for polygon clipping. However, when a closed polygon is clipped as a collection of lines with line clipping algorithm, the original closed polygon becomes one or more open polygon or discrete lines as shown in the Fig. 5.22. Thus, we need to modify the line clipping algorithm to clip polygons.

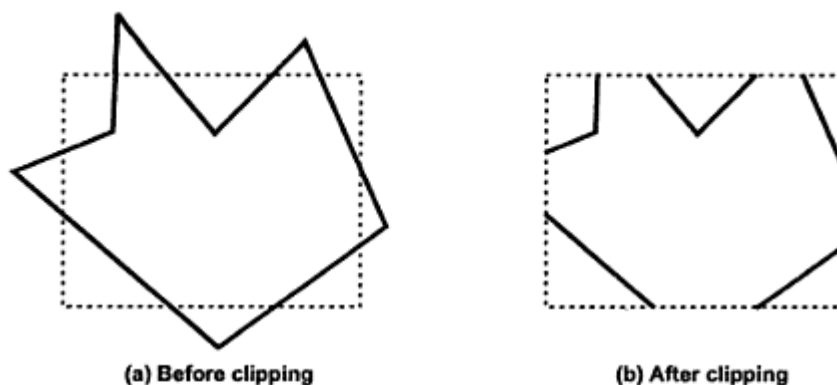


Fig. 5.22 Polygon clipping done by line clipping algorithm

We consider a polygon as a closed solid area. Hence after clipping it should remain closed. To achieve this we require an algorithm that will generate additional line segment which make the polygon as a closed area. For example, in Fig. 5.23 the lines a - b, c - d, d - e, f - g, and h - i are added to polygon description to make it closed.

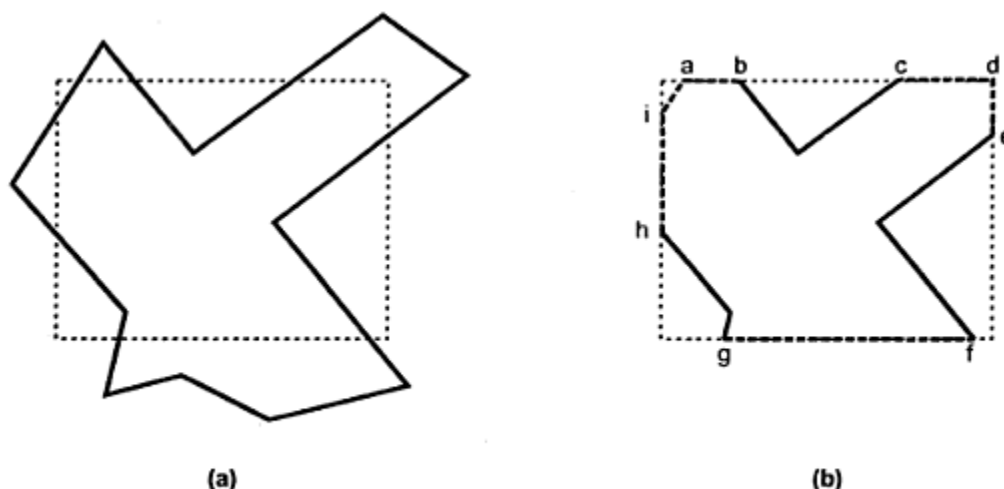


Fig. 5.23 Modifying the line clipping algorithm for polygon

Adding lines c - d and d - e is particularly difficult. Considerable difficulty also occurs when clipping a polygon results in several disjoint smaller polygons as shown in the Fig. 5.24. For example, the lines a - b, c - d, d - e and g - f are frequently included in the clipped polygon description which is not desired.

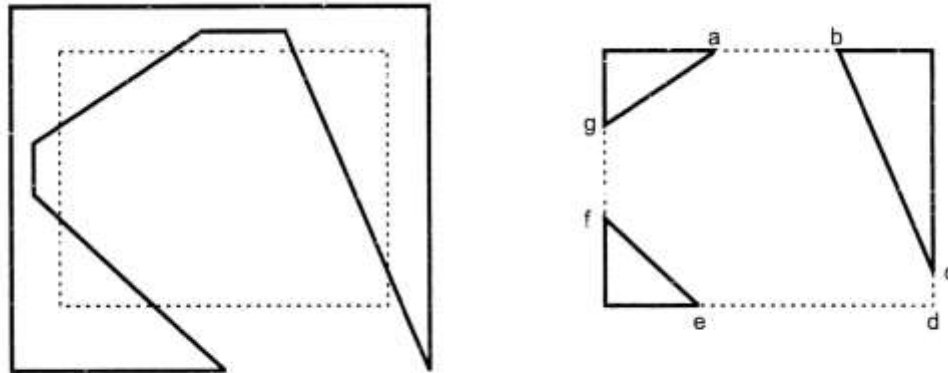


Fig. 6.21 Disjoint polygons in polygon clipping

6.5 Sutherland - Hodgeman Polygon Clipping

A polygon can be clipped by processing its boundary as a whole against each window edge. This is achieved by processing all polygon vertices against each clip rectangle boundary in turn. Beginning with the original set of polygon vertices, we could first clip the

polygon against the left rectangle boundary to produce a new sequence of vertices. The new set of vertices could then be successively passed to a right boundary clipper, a top boundary clipper and a bottom boundary clipper, as shown in Fig. 6.22. At each step a new set of polygon vertices is generated and passed to the next window boundary clipper. This is the fundamental idea used in the Sutherland - Hodgeman algorithm.

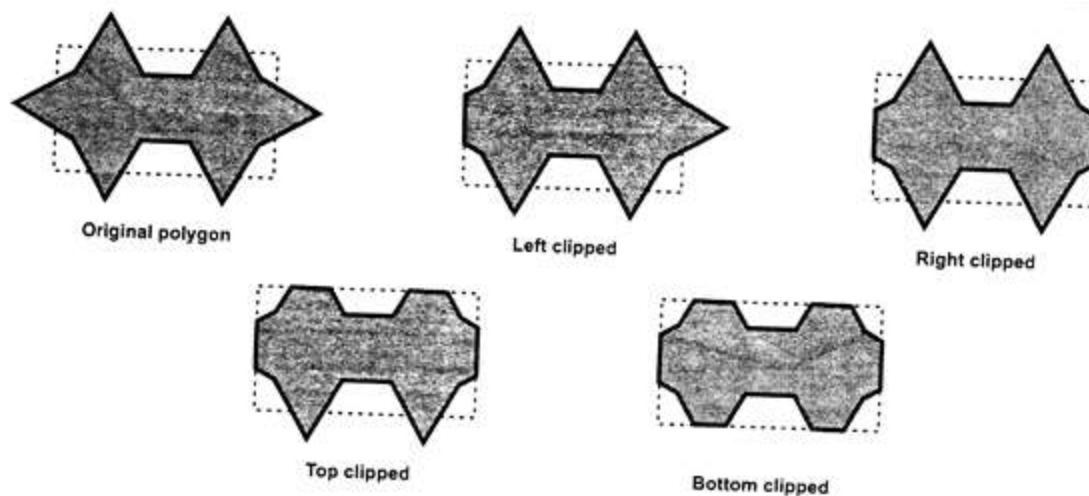


Fig. 6.22 Clipping a polygon against successive window boundaries

The output of the algorithm is a list of polygon vertices all of which are on the visible side of a clipping plane. Such each edge of the polygon is individually compared with the clipping plane. This is achieved by processing two vertices of each edge of the polygon around the clipping boundary or plane. This results in four possible relationships between the edge and the clipping boundary or plane. (See Fig. 6.23).

1. If the first vertex of the edge is outside the window boundary and the second vertex of the edge is inside then the intersection point of the polygon edge with the window boundary and the second vertex are added to the output vertex list (See Fig. 6.23 a).
2. If both vertices of the edge are inside the window boundary, only the second vertex is added to the output vertex list. (See Fig. 6.23 b).
3. If the first vertex of the edge is inside the window boundary and the second vertex of the edge is outside, only the edge intersection with the window boundary is added to the output vertex list. (See Fig. 6.23 c).
4. If both vertices of the edge are outside the window boundary, nothing is added to the output list. (See Fig. 6.23 d).

Once all vertices are processed for one clip window boundary, the output list of vertices is clipped against the next window boundary.

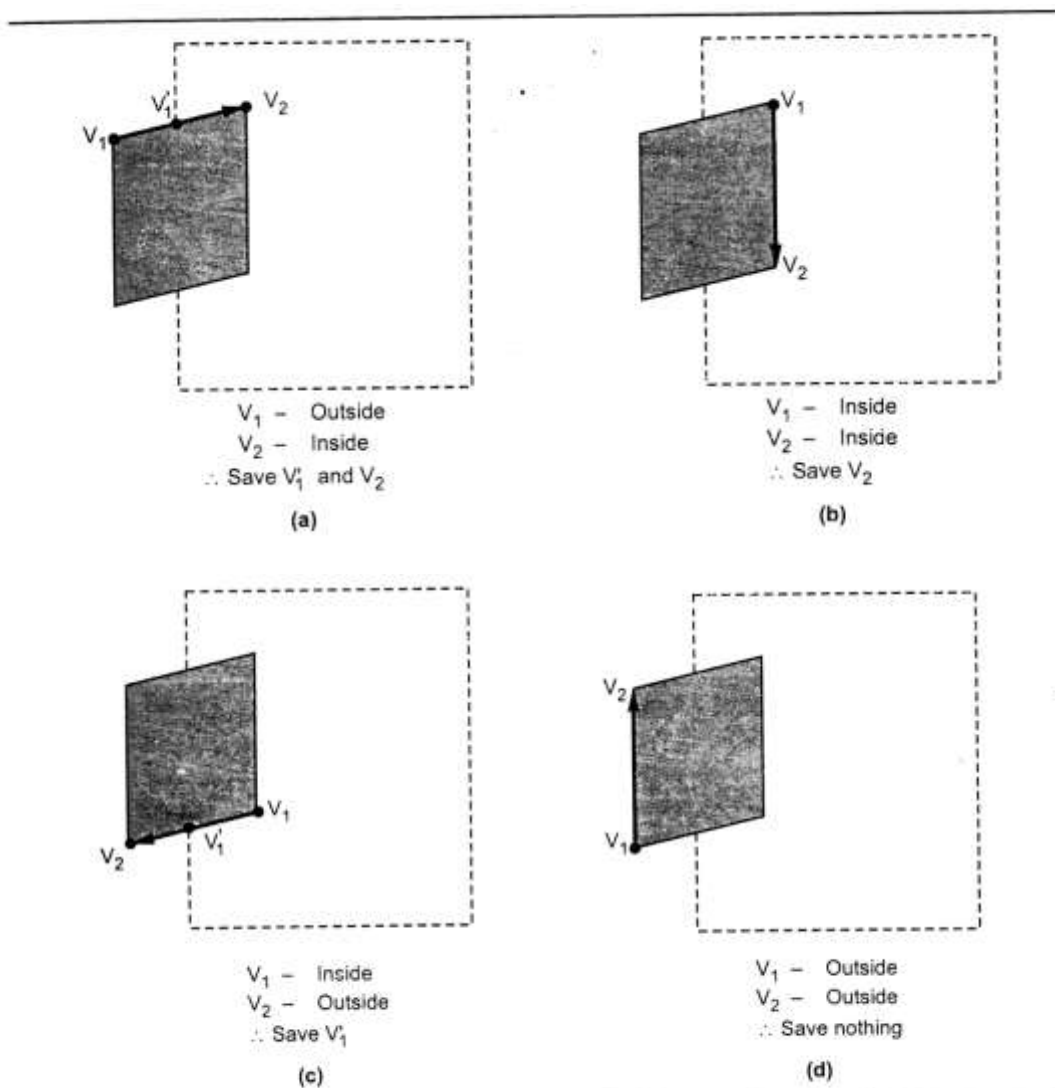


Fig. 6.23 Processing of edges of the polygon against the left window boundary

Going through above four cases we can realize that there are two key processes in this algorithm.

1. Determining the visibility of a point or vertex (Inside - Outside test) and
2. Determining the intersection of the polygon edge and the clipping plane.

One way of determining the visibility of a point or vertex is described here. Consider that two points A and B define the window boundary and point under consideration is V, then these three points define a plane. Two vectors which lie in that plane are AB and AV. If this plane is considered in the xy plane, then the vector cross product $AV \times AB$ has only a z component given by $(x_V - x_A)(y_B - y_A) - (y_V - y_A)(x_B - x_A)$. The sign of the z component decides the position of point V with respect to window boundary.

If z is : Positive - Point is ~~on~~ the right side of the window boundary
 Zero - Point is ~~on~~ the window boundary
 Negative - Point is on the left side of the window boundary

Ex. 6.6 : Consider the clipping boundary as shown in the Fig. 6.24 and determine the positions of points V_1 and V_2 .

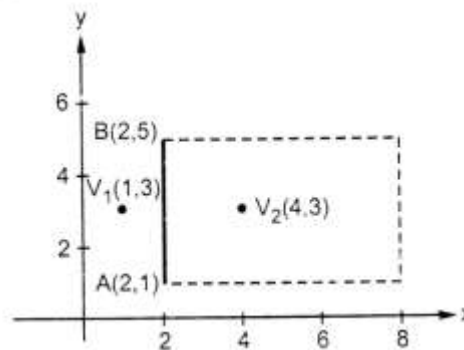


Fig. 6.24

Sol. : Using the cross product for V_1 we get,

$$\begin{aligned} & (x_V - x_A)(y_B - y_A) - (y_V - y_A)(x_B - x_A) \\ &= (1 - 2)(5 - 1) - (3 - 1)(2 - 2) \\ &= (-1)(4) - 0 \\ &= -4 \end{aligned}$$

The result of the cross product for V_1 is negative hence V_1 is on the left side of the window boundary.

$$\begin{aligned} & \text{Using the cross product for } V_2 \text{ we get, } (4 - 2)(5 - 1) - (3 - 1)(2 - 2) \\ &= (2)(4) - 0 \\ &= 8 \end{aligned}$$

The result of the cross product for V_2 is positive hence V_2 is on the right side of the window boundary.

The second key process in Sutherland - Hodgeman polygon clipping algorithm is to determine the intersection of the polygon edge and the clipping plane. Any of the line intersection (clipping) techniques discussed in the previous sections such as Cyrus-Beck or mid point subdivision can be used for this purpose.

Sutherland-Hodgeman Polygon Clipping Algorithm

1. Read coordinates of all vertices of the polygon.
2. Read coordinates of the clipping window
3. Consider the left edge of the window
4. Compare the vertices of each edge of the polygon, individually with the clipping plane
5. Save the resulting intersections and vertices in the new list of vertices according to four possible relationships between the edge and the clipping boundary discussed earlier.
6. Repeat the steps 4 and 5 for remaining edges of the clipping window. Each time the resultant list of vertices is successively passed to process the next edge of the clipping window.
7. Stop.

Ex. 6.7 : For a polygon and clipping window shown in Fig. 6.25 give the list of vertices after each boundary clipping.

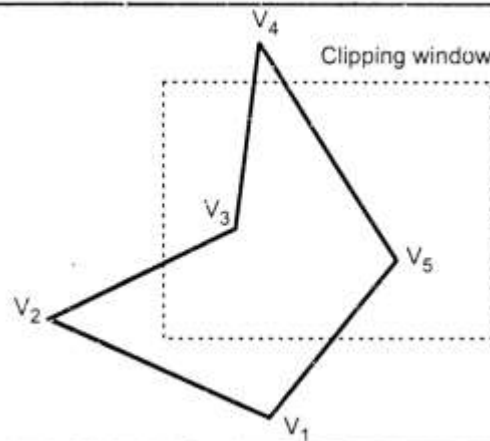


Fig. 6.25

Sol. : Original polygon vertices are V_1, V_2, V_3, V_4, V_5 . After clipping each boundary the new vertices are given in Fig. 6.26

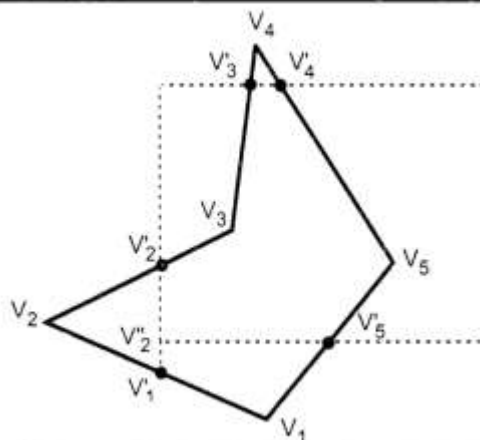


Fig. 6.26

After left clipping : $V_1, V'_1, V'_2, V_3, V_4, V_5$
 After right clipping : $V_1, V'_1, V'_2, V_3, V_4, V_5$
 After top clipping : $V_1, V'_1, V'_2, V_3, V'_3, V'_4, V_5$
 After bottom clipping : $V''_2, V'_2, V_3, V'_3, V'_4, V_5, V'_5$